

计算机硬件从零到整机

从电、二极管、逻辑门和时钟到最小 CPU 与整机硬件

CPU Books

本书沿着器件、电平、逻辑、状态、时钟、算术、数据通路、控制器、最小 CPU、主存、总线和 I/O 的顺序，把一台计算机从最小可理解部件推导出来。

前言

这本书从硬件本身开始。全书首先建立底层合同：一根线上连续变化的电压怎样被稳定解释成 0 和 1；一个器件怎样让电流更容易沿某个方向通过；一个受控开关怎样让一个信号决定另一条电流路径是否导通。只有这些合同成立，后续的逻辑门、寄存器、ALU、CPU 和整机接口才具有可靠含义。

本书按照四个较厚章节展开。第一章把电平、器件、开关、逻辑门和组合电路放在同一条链路中讨论，说明 bit 如何从物理电平进入可计算的逻辑网络。第二章集中讨论状态、时钟与同步边界，说明硬件如何保存过去，并在清楚的时钟边界提交下一值。

第三章讨论数值解释、算术电路、ALU、数据通路和控制器。它把固定宽度 bit、补码、标志位、选择器、寄存器阵列和控制 trace 统一起来，使读者能够从旧状态走读到新状态。第四章再把这些部件合成为 CPU，并继续连接主存、cache、总线、I/O、DMA、中断和启动硬件，同时借助 8086、80386 等经典芯片观察真实处理器的历史剖面。

这种组织方式刻意避免把 CPU 作为孤立名词提前给出。CPU 在本书中是前述硬件部件按同步边界组织后的结果：PC 保存下一条编码地址，译码逻辑产生控制线，数据通路执行计算和写回，外部接口把处理器连接到主存与设备。读完整本书后，读者应能把一台机器从电平、开关、门电路、组合逻辑、时序逻辑、运算部件、数据通路、控制器一路连到整机。

本书采用接近 CSAPP (Computer Systems: A Programmer's Perspective) 等经典系统教材的标注方式：每讲一个抽象，都同时给出机器层证据。读逻辑门时同时检查真值表和开关路径；读组合逻辑时同时检查函数、延迟和毛刺；读寄存器时同时检查 bit 和采样边界；读 CPU 时同时检查指令、控制信号和数据通路 trace。章节末尾的“经典教材标注”不是附加材料，而是用于检验硬件理解是否已经落到证据层面。

目录

第一部分 器件、开关与组合逻辑	1
第 1 章 电平、开关、逻辑门与组合电路	3
一、从一根线得到可靠 bit	3
二、从器件到受控电流路径	5
三、从路径表得到逻辑门	6
四、门网络构成组合逻辑	8
五、延迟、负载、毛刺与验收	9
第二部分 状态、时钟与同步电路	11
第 2 章 状态、时钟与同步边界	13
一、反馈、锁存器与保存一个 bit	13
二、触发器、时钟周期与采样窗口	13
三、关键路径、复位与跨边界输入	14
四、寄存器、计数器与移位结构	15
五、状态机把硬件动作切成阶段	15
第三部分 运算部件与数据通路	17
第 3 章 数值、ALU、数据通路与控制器	19
一、固定宽度 bit 的数值解释	19
二、ALU 把候选计算收进一个组合核心	20
三、数据通路让值从旧状态走到新状态	21
四、控制器输出一组一致的控制线	22
五、用控制 trace 走读一次动作	23
第四部分 最小 CPU 与整机硬件	25
第 4 章 CPU、经典芯片与整机硬件	27
一、从动作编码到最小 CPU	27
二、8086：从芯片接口看早期微处理器	28
三、80386：32-bit、保护模式与地址转换	29
四、主存、总线、I/O 与 DMA	29
五、整机启动与经典芯片后的演进	30

器件、开关与组合逻辑

本部分导读

本部分集中回答一个问题：怎样从可控电流路径得到稳定的 0 和 1，再把多个 0/1 组合成可计算的逻辑网络。讨论对象是器件、电平、门电路和组合逻辑。

电平、开关、逻辑门与组合电路

本章将“电和器件”“开关到门”“组合逻辑”合并为一条完整链路：连续变化的电压先被解释成稳定的 0/1；二极管和受控开关提供可预测的电流路径；开关网络形成 NOT、NAND、NOR、XOR 等门；门再组合成判断器、加法器、选择器和译码器。这样的组织方式有助于读者在同一章内建立从物理电平到组合电路的连续推导。

本章不展开完整的半导体物理，也不把内容处理成布尔代数速查表。它关注一个核心问题：一根真实导线上的电压，怎样逐步成为 CPU 可使用的 bit 计算。后续所有寄存器、ALU、数据通路、控制器、内存和 I/O，最终都必须满足这里建立的底层合同。

核心思想

数字硬件的第一层抽象不是门，而是带余量的电平；数字硬件的第一层实现不是公式，而是受控电流路径。

一、从一根线得到可靠 bit

电路里说“一根线是 1”，完整含义不是“线上写着数字 1”，而是“这个节点相对于参考地的电压落在接收端承认的高电平区间内”。节点可以理解成多条金属连接汇合成的同一个电气位置；参考地是整块电路共同约定的 0V 参考点；电压是两个点之间的差值。数字电路里说某根信号线是 0V、3.3V 或 5V，实际都是在说它相对于地的电压。

电流说明电荷正在沿通路流动。若输出节点被一条通路接向电源，它可能被拉到高电平；若被一条通路接向地，它可能被拉到低电平。若两条通路都断开，节点不会自动变成 0，而是可能暂时停在旧电压附近，再被漏电流、干扰、后级输入或测量探针慢慢推走。若上拉和下拉两条强通路同时打开，电源到地之间形成冲突电流，电平不可靠，还会增加功耗和发热。

需要先区分导线和程序变量。变量没有被赋值时，语言可以规定默认值；导线没有被驱动时，只剩物理过程。它可能因为寄生电容保存旧电压，表现为保留上一次状态；也可能被旁边切换的信号线、人体接近、探针输入阻抗或后级漏电流轻微影响。调试数字硬件时，不能把这种偶然稳定误认为设计正确。

以按钮输入为例。假设按钮按下表示“请求发生”。一种不完整的接法是按钮一端接信号线，另一端接电源：

```
button pressed -> signal connected to VCC
button released -> signal connected to nothing
```

按下时，信号线被电源路径拉高，该部分行为符合预期。松开时，信号线并没有被明确拉低，它只是断开了。这个节点可能保留先前电荷，也可能被旁边导线耦合出一定电压，还可能被后级输入漏电流拖向某个不确定位置。于是接收端可能读到 1、0 或过渡电压。问题的根源不是程序错误，而是电路没有为节点规定明确归宿。

正确的第一层想法是：任何要被后级判断的输出节点，都必须在每一种输入情况下被明确地拉向高电平或低电平。按钮松开时需要一条弱下拉路径把信号线拉向地；按钮按下时，按钮路径把它拉向电源。弱下拉不是为了在按钮按下时和按钮争夺电源，而是为了在按钮松开时给节点一个默认归宿。它太弱，节点下降慢、抗干扰差；它太强，按钮按下时浪费电流，甚至让高电平不够高。

场景	高电平路径	低电平路径	结论
----	-------	-------	----

按下	按钮把节点接向电源	弱下拉仍存在	可靠高
松开	高电平路径断开	弱下拉把节点接向地	可靠低
接触抖动	高电平路径快速断续	弱下拉持续存在	需要消抖
导线断裂	按钮路径永远断开	若下拉还在，则为低	有默认值
没有下拉	高电平路径断开	低电平路径也断开	悬空

该表给出硬件读法的雏形。它不只问逻辑上应该是 0 还是 1，还问路径在哪里、谁驱动谁、异常时会落到哪个默认状态。后面讲复位、总线仲裁和设备中断时，同样要用这种检查法：每个状态都要有明确来源，每个默认值都要有物理路径支持。

电平也不能理解成两个数学点。假设一块小电路用 5V 供电。线上的真实电压可能是 0V、0.17V、1.3V、2.6V、4.8V，也可能在切换过程中从 0V 慢慢升到 5V。硬件不能要求每一级都精确读出这个连续数值。更可靠的做法是规定两个稳定区间：低到足够接近地电位时解释为 0，高到足够接近电源电位时解释为 1，中间一段不作为长期稳定状态。

电压范围	逻辑解释	设计意义
接近地电位	逻辑 0	小噪声不会轻易把它推成 1
中间区域	不作为稳定状态	切换可以经过，但不能长期停留
接近电源电位	逻辑 1	小噪声不会轻易把它拉成 0

可以用四个电压指标描述两级电路之间的合同。 V_{OH} 是输出端保证的高电平下限， V_{OL} 是输出端保证的低电平上限； V_{IH} 是输入端承认为高的最低电压， V_{IL} 是输入端承认为低的最高电压。若一个电路保证 V_{OH} 高于下一级需要的 V_{IH} ，中间差额就是高电平噪声余量；若 V_{OL} 低于下一级允许的 V_{IL} ，中间差额就是低电平噪声余量。

符号	含义	读法
V_{OH}	输出保证为高时的最差电压	前一级说：我给出的 1 至少高到这里
V_{IH}	输入承认为高的最低电压	后一级说：至少给到这里，我才认作 1
V_{OL}	输出保证为低时的最差电压	前一级说：我给出的 0 至多高到这里
V_{IL}	输入承认为低的最高电压	后一级说：低到这里以下，我才认作 0

数字抽象并不是忽略模拟世界，而是把模拟世界约束在合同之内。只要所有连接都满足这些最差条件，后续讨论 bit、真值表和指令编码才有可靠基础。若这些条件不满足，系统层面可能出现间歇性现象：同一块板有时能启动、有时不能；按键有时触发两次；某条总线在高温下出错；更换更长排线后原本稳定的电路开始误判。这些现象不一定来自逻辑表达式错误，也可能来自电平合同被破坏。

输出还必须能驱动下一级。一个节点如果只能在空载时看起来像 1，一接上后级就被拖低，它不是可靠数字输出。后级输入、板级走线、封装和探针都有寄生电容。把一个节点从 0 拉到 1，本质上是在给这个电容充电；把它从 1 拉到 0，本质上是在放电。驱动路径越弱，等效电阻越大；负载越多，等效电容越大；上升和下降越慢。

```
larger load capacitance -> slower edge
weaker driver           -> slower edge
slower edge             -> less time margin before sampling
```

所以示波器上看到的真实信号不是理想直角。电压边沿有斜率，可能有过冲、振铃、平台和噪声。逻辑 0/1 的背后是一条随时间变化的电压曲线，接收端只是在某个时刻把它归类为 0 或 1。若这个时刻到来之前电压还没有进入可靠区间，最终会

稳定也不够。

二、从器件到受控电流路径

二极管可以先理解成方向性很强的器件。在合适偏置下，它比较容易让电流沿一个方向通过；反向偏置时，它基本阻止电流通过。这个模型不需要一开始展开半导体物理，却已经足够说明硬件设计的一条基本路线：利用器件物理特性，制造可预测的电流路径。

二极管的“单向”不是理想数学开关。正向导通通常需要一定压差，导通后还有压降；反向并非绝对没有电流，而是漏电很小；电压过高时还可能击穿。初读数字硬件时可以先用理想模型建立方向感，但不能忘记真实器件会吃掉电压余量、引入延迟和功耗。

二极管事实	对数字电路的影响	读书时要追问
有方向性	可以限制电流路径	这条路径在何种输入下导通
有正向压降	输出电平会损失一部分余量	级联后高电平还够不够高
有漏电和极限	不能把反向阻断看成无限好	极端温度、电压下是否仍可靠
不能主动放大	不能恢复变弱的电平	下一级是否需要缓冲或开关恢复

二极管逻辑可以表达某些简单关系。若有两路输入想共同点亮一个指示节点，任意一路为高时都能通过二极管把输出节点拉高，输出就表现出 OR 的味道：

```
if A can drive through a diode:
    Y may be pulled high
if B can drive through a diode:
    Y may be pulled high
```

该例能够说明方向性逻辑，但也暴露出局限。第一，二极管会带来压降，输出高电平可能比输入高电平低一截；级联多了，电平余量会越来越紧。第二，二极管只能提供方向性，不能主动把一个偏弱的高电平重新恢复成强高电平。第三，它不方便实现取反，而取反是构造通用逻辑必须具备的能力。

取反为什么重要？因为许多条件天然需要反向解释。传感器未触发时允许运行，触发时禁止运行；按钮未按下时保持，按下时改变。若电路只能表达“有一路导通”，却不能把 1 变成 0、把 0 变成 1，那么可组合的逻辑很快就受限。要继续向前，需要一个信号能控制另一条电流路径是否导通。

晶体管的细节很深，但作为第一层硬件模型，可以先把它当成受控开关。控制端不是直接给输出供电，而是决定输出节点到某条电源路径之间是否接通。受控开关和普通机械开关最大的差别，是控制信号本身也是电信号。按钮需要人按，晶体管可以由前一级逻辑输出控制。于是硬件有了“信号控制信号”的递归能力：一个门的输出可以控制下一批开关，下一批开关再形成新的输出。

反相器可以作为第一种完整逻辑门来理解。输入低时，上拉路径导通、下拉路径断开，输出被拉高；输入高时，上拉路径断开、下拉路径导通，输出被拉低。注意这里不是“输入低所以输出自动高”，而是“输入低使某个高电平路径导通”。把每一句逻辑描述翻译成路径描述，能避免很多硬件误解。

输入	上拉路径	下拉路径	输出
低电平	导通	断开	被拉高
高电平	断开	导通	被拉低
过渡中	可能部分导通	可能部分导通	暂时不应采样

第三行很重要。真实输入从低到高不是瞬间跳变，在过渡区间里，上拉和下拉可能都不处于理想开/关状态。短时间内出现电源到地的电流并不奇怪，这就是动态功

耗和短路功耗的一部分。只要过渡足够快、采样避开中间区域，数字抽象仍然成立。若输入边沿太慢，电路可能在中间区域停留过久，功耗和误判风险都会上升。

把开关串联，可以表达“两个条件都满足才有通路”；把开关并联，可以表达“任意一个条件满足就有通路”。以下拉网络为例，两个受控开关串联，只有 A 和 B 都让开关导通时，输出节点才会被拉向低电平；两个开关并联，只要 A 或 B 有一个导通，输出节点就会被拉低。上拉网络通常要和下拉网络互补，保证输出在稳定输入下要么被拉高，要么被拉低，而不是两边都断或两边都强通。这种“互补路径”思想，后面会自然走到 CMOS 逻辑。

三、从路径表得到逻辑门

门电路的第一步不是画符号，而是把输入组合列完整。两个输入 A、B 只有四种组合。电路必须对每一种组合给出明确输出，而且每一行都要满足前面的要求：输出节点不能悬空，不能上下冲突，必须能在规定时间内进入可靠电平区间。

以安全允许灯为例。A 表示保护盖合上，B 表示急停未按下。允许灯亮的条件很严格：任一条件不满足，输出都必须为 0。

A	B	允许灯	为什么
0	0	0	盖子没合上，急停也不是可运行状态
0	1	0	急停条件满足，但盖子没合上
1	0	0	盖子合上了，但急停条件不满足
1	1	1	两个安全条件同时满足

该表对应 AND 的行为。它不是抽象数学定义，而是“两个条件都满足才允许”的硬件约束。若某个实现让 A=1、B=0 时灯也亮，那就不是 AND 近似得不够好，而是安全行为错了。真值表的价值在于防止“只凭自然语言写电路”。“两个条件都满足”看起来简单，但实现时很容易漏掉某一行。

再看故障灯。A 表示温度报警，B 表示电流报警。故障灯的规则反过来：只要有一个坏消息出现，就要亮。

A	B	故障灯	为什么
0	0	0	没有任何故障证据
0	1	1	电流报警已经足够触发故障
1	0	1	温度报警已经足够触发故障
1	1	1	两个故障同时存在，仍应报警

该表对应 OR 的行为。AND 和 OR 的差别不在符号长什么样，而在需求不同：一个要求所有条件同时成立，一个要求任意证据成立。NOT 只有一个输入，但同样要列完整。若 A 表示“禁止信号”，输出 Y 表示“允许动作”，那么 A=0 时 Y=1，A=1 时 Y=0。它解决的是反向解释：输入出现时，输出反而关闭。

不要把“0 是假、1 是真”机械套到每个硬件信号上。某些信号是低有效，例如 `reset_n`、`chip_select_n`，低电平才表示动作发生。读这类信号时，先写真值表，再谈门电路。

真值表说清楚“应该怎样”，路径表说明“为什么会这样”。用受控开关看 AND，最自然的结构是串联。两个开关串在同一条路径上，只有 A 和 B 都让自己的开关闭合时，整条路径才通。OR 更像并联。A 控制一条路径，B 控制另一条路径。只要任意一路闭合，输出就能被拉向目标电平。

A	B	串联路径状态	AND 输出为什么成立或不成立
0	0	两个开关都断开	没有完整导通路径
0	1	A 断开, B 闭合	串联路径仍被 A 切断
1	0	A 闭合, B 断开	串联路径仍被 B 切断
1	1	两个开关都闭合	从源到输出的路径完整

如果只画一条“让输出为 1 的路径”，还没有完成门电路设计。输出节点不能靠悬空表示 0。因此每一行真值表还要补一张路径表，说明高电平路径和低电平路径的状态。一个合法门在稳定输入下，不应该让输出悬空，也不应该让上拉和下拉强路径同时导通。

从开关网络看，NAND 和 NOR 往往比 AND 和 OR 更像基础积木。原因是它们天然带反相输出，更容易形成互补的上拉和下拉结构：当下拉网络导通时输出被拉低；当下拉网络不导通时，上拉网络负责把输出拉高。

A	B	NAND 输出	开关直觉
0	0	1	下拉串联路径断开，上拉负责拉高
0	1	1	A 断开，下拉路径不完整
1	0	1	B 断开，下拉路径不完整
1	1	0	A、B 都闭合，下拉路径完整，输出被拉低

NOR 可以用同样方式读。NOR 的输出只有在 $A=0$ 且 $B=0$ 时为 1；只要 A 或 B 为 1，输出就为 0。若以下拉网络看，A、B 并联，只要任一路输入为 1，就能把输出拉低；只有两路都不导通时，上拉网络把输出拉高。

只用 NAND 就能构造 NOT、AND、OR：

```
NOT A = A NAND A
A AND B = (A NAND B) NAND (A NAND B)
A OR B = (A NAND A) NAND (B NAND B)
```

这说明 NAND 在功能上完备，不说明所有实现都应该只用 NAND。真实设计会在功能正确的前提下，考虑门级数量、路径延迟、面积、功耗和负载。逻辑式等价，只说明稳定输出行为相同；它不保证延迟、功耗和驱动能力相同。

XOR 的输出在两个输入不同的时候为 1，相同的时候为 0。它解决的是“检测两个 bit 是否不同”这个具体问题。

A	B	A XOR B	解释
0	0	0	两个输入相同
0	1	1	两个输入不同
1	0	1	两个输入不同
1	1	0	两个输入相同

可以把 XOR 拆成两种会输出 1 的情况： $A=1$ 且 $B=0$ ，或者 $A=0$ 且 $B=1$ ：

```
A XOR B = (A AND NOT B) OR (NOT A AND B)
```

这条式子不要只当代数记忆。它对应两条证据路径：第一条证明“只有 A 为 1”，第二条证明“只有 B 为 1”。最后的 OR 合并这两条证据。若两个都为 0，没有证据成立；若两个都为 1，两条“只有一个为 1”的证据也都不成立。XOR 后面会反复出现：两个 bit 相加时，和位就是“两个输入是否不同”；比较两个 bit 时，XOR 可以告诉我们它们是否不同；接一个 NOT，就能得到“两个输入是否相同”。

四、门网络构成组合逻辑

组合逻辑的特点很直接：输出只由当前输入决定，不保存过去。它像一个硬件函数，但这个函数不是一步一步执行的过程，而是一张由门和连线组成的网络。输入变化后，信号沿多条路径同时传播；等延迟过去，输出稳定到当前输入对应的值。

先做一个三传感器判断器。输入 A、B、C 表示三个传感器，要求至少两个传感器为 1 时输出 1。这个需求不能只凭直觉画线，先列出行为：

A	B	C	Y	原因
0	0	0	0	一个条件都没有
1	0	0	0	只有一个条件
0	1	0	0	只有一个条件
0	0	1	0	只有一个条件
1	1	0	1	A 和 B 同时满足
1	0	1	1	A 和 C 同时满足
0	1	1	1	B 和 C 同时满足
1	1	1	1	三个都满足

从表中可以直接得到一个门网络：

$$Y = (A \text{ AND } B) \text{ OR } (A \text{ AND } C) \text{ OR } (B \text{ AND } C)$$

这条表达式里有三条“证据路径”。第一条路径证明 A、B 同时为 1；第二条路径证明 A、C 同时为 1；第三条路径证明 B、C 同时为 1。最后的 OR 不是任意合并，而是在检查三条证据中是否至少有一条成立。按这种方式阅读，门网络就不只是公式变形，而是把真值表中每一种输出为 1 的原因接成硬件。

两个 bit 相加是第二个基本组合逻辑例子。输出不止一个 bit，因为 1+1 得到二进制 10，需要一个和位 S 和一个进位 C。

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

观察表格可以看到，S 在 A、B 不同时为 1，C 在 A、B 同时为 1：

$$\begin{aligned} S &= A \text{ XOR } B \\ C &= A \text{ AND } B \end{aligned}$$

这就是半加器。它已经完成了一位相加，但它还缺一个输入：来自低位的进位。只要要做多位加法，最低位之外的每一位都不能只看 A 和 B，还要看低一位是否进上来。

全加器输入为 A、B、Cin，输出为 S、Cout。可以先把 A 和 B 用半加器相加，得到临时和 S1 和进位 C1；再把 S1 和 Cin 用第二个半加器相加，得到最终和 S 和第二个进位 C2；最后把两个进位合并。

$$\begin{aligned} S1 &= A \text{ XOR } B \\ C1 &= A \text{ AND } B \\ S &= S1 \text{ XOR } Cin \\ C2 &= S1 \text{ AND } Cin \\ Cout &= C1 \text{ OR } C2 \end{aligned}$$

把全加器一位一位串起来，就得到串行进位加法器。它的逻辑很清楚：第 0 位算出进位，传给第 1 位；第 1 位再把进位传给第 2 位。问题也很清楚：若最低位

产生的进位一路传到最高位，最高位必须等前面所有进位稳定后才能稳定。这是组合逻辑里第一次真正遇到“逻辑正确”和“速度足够”的差别。真值表保证最后结果正确，传播延迟决定结果多久之后才可靠。

多路选择器解决“多个输入选一个输出”。2 选 1 选择器的行为是：

```
if sel = 0, Y = A
if sel = 1, Y = B
```

上述 if 只是描述行为，电路本身没有顺序执行。它的门级表达式是：

```
Y = (NOT sel AND A) OR (sel AND B)
```

选择器的作用很基础。若一条输出线可能来自加法器，也可能来自传感器，也可能来自某个寄存器，那么必须有一个明确的选择信号决定哪一路接到输出。没有选择器，多路来源同时驱动同一条线就会冲突。

译码器做相反方向的事：把一个二进制编号展开成 one-hot 信号。2-bit 输入可以选中 4 条输出中的一条。

输入	输出
00	Y0=1, Y1/Y2/Y3=0
01	Y1=1, 其他为 0
10	Y2=1, 其他为 0
11	Y3=1, 其他为 0

选择器把多路收成一路，译码器把编号展开成多路选择。后面无论是寄存器阵列、数据通路，还是存储器地址选择，都会反复看到这两个结构。

五、延迟、负载、毛刺与验收

门电路在纸上像瞬间给出输出的函数，真实硬件却至少有四个限制。第一，输入变化后，输出需要传播延迟才稳定。第二，一个输出能驱动的后级输入数量有限，后级太多会让切换变慢，甚至让电平余量变差。第三，门的输出必须重新形成清楚的 0/1，不能只靠一个偏弱路径勉强改变节点电压。第四，多条路径延迟不同，输入变化过程中可能出现短暂毛刺。

限制	含义	后续影响
传播延迟	输出不会在输入变化的同一瞬间稳定	组合逻辑层数越多，稳定越慢
扇出	一个输出只能可靠驱动有限后级	大网络需要缓冲器和分层布线
电平恢复	输出要重新形成清楚 0/1	不能只追求逻辑表达式正确
毛刺	不同路径到达时间不同	后级不能在毛刺期间采样

考虑一个反例。若把十几个门的输入全接到同一个弱输出上，真值表仍然可以保持正确，但物理电路可能切换缓慢。输出从 0 变 1 时，要给所有后级输入电容充电；如果下一级在它尚未进入可靠高电平区间时读取，就可能看到旧值或不确定值。因此，分析组合逻辑时不能只数门的个数，还要考察最长路径、负载和稳定时间。

短暂毛刺也很常见。假设一个输出由多条逻辑路径合成，输入同时变化时，每条路径稳定的时间可能不同。真值表只告诉我们变化前和变化后的稳定结果，却不保证中间不会短暂出现错误电平。如果后级只是另一个组合门，毛刺可能很快消失；如果后级在毛刺期间被时钟采样，错误就可能被保存下来。后面讲触发器和时钟时，会把这个问题放大讨论。

还要区分反相器和缓冲器。反相器改变逻辑含义，缓冲器通常保持逻辑含义但增

强驱动能力或隔离负载。很多时候，插入缓冲器不是为了“多一个门”，而是为了让一个信号能推得动更长的线或更多的后级。软件读者容易觉得保持值不变的缓冲器没有意义，但在硬件里，驱动能力本身就是功能的一部分。

到这里，本章完成了从电平到组合逻辑的闭环：节点要有明确驱动路径；电平要有噪声余量；二极管提供方向性但不能恢复电平；受控开关让信号控制电流路径；路径表和真值表共同定义门；门网络构成加法器、选择器、译码器等组合电路；组合电路虽然没有记忆，却有延迟、负载和毛刺。下一章开始讨论另一件事：怎样让硬件保存状态，并按时钟边界一步一步推进。

本章的验收标准不是“会背 AND、OR、NOT、XOR 的真值表”，而是能把每个抽象还原成机器证据。

- 电平证据：前一级的最差高/低输出，必须落在后一级输入阈值允许的范围内，并留下噪声余量。
- 路径证据：任何输出节点都要能写成“高电平路径是否导通、低电平路径是否导通、是否存在冲突电流、是否存在悬空”四项。
- 门级证据：任选一个二输入门，列出 4 行真值表，再为每一行写出上拉网络和下拉网络状态。
- 组合证据：半加器的 Sum 是 XOR，Carry 是 AND；全加器是在同一张真值表中纳入进位输入后的组合电路。
- 延迟证据：同一个逻辑函数可以有不同门级网络，最长路径不同，稳定时间也不同。
- 负载证据：一个门的输出不是无限共享变量，后级数量、线长和缓冲策略会改变电气行为。
- 检查题：若组合网络输出短暂从 0 跳到 1 又回到 0，而最终值正确，它是否一定构成设计错误？答案取决于后级是否在毛刺期间采样。

经典系统教材常把抽象位函数和实际机器证据放在同一页上看。只看位函数会漏掉电平、驱动和时序；只看器件又会失去逻辑行为。两者合在一起，才是硬件设计对象。

状态、时钟与同步电路

本部分导读

组合逻辑没有记忆。要让硬件按阶段推进，必须引入反馈、锁存器、触发器、时钟、复位、寄存器、计数器和状态机。本部分说明状态怎样稳定保存、怎样按边界更新。

状态、时钟与同步边界

第一章讲的是组合逻辑：输入稳定后，输出由当前输入唯一决定。它能计算，却不能保存过去。真正的机器必须能把某个结果留下来，等下一个步骤继续使用；也必须能规定“什么时候更新状态”，否则反馈路径会让硬件在一个周期内连续变化，机器无法分步骤推进。

本章将反馈、锁存器、触发器、时钟、复位、寄存器、计数器和状态机合并讨论。读完后，应当能把“状态”理解成真实硬件边界：一些 bit 在时钟边界被提交，边界之间由组合逻辑准备下一值。

核心思想

组合逻辑负责准备下一值，状态元件负责在边界保存下一值。同步硬件就是把这两件事反复组织起来。

一、反馈、锁存器与保存一个 bit

保存一个 bit 的起点是反馈。两个反相器首尾相接，会形成两个稳定状态。若左边节点为 0，右边经过反相后为 1；右边的 1 再反相回来，正好维持左边为 0。反过来，左边为 1、右边为 0 也能稳定。只要没有外部强制改变，这个状态会保持。

该结构说明，“保存一个 bit”对应电路中某些节点在反馈作用下停留于稳定物理状态。一个稳定状态解释为 0，另一个稳定状态解释为 1。状态的本质，是电路有能力在输入不再变化时仍然维持某个内部结果。

但单纯反馈还不足以形成可用存储单元。若无法可靠地把它从 0 改成 1，或从 1 改成 0，它只是一个稳定反馈环。因此需要写入控制。锁存器可以理解为带写使能的 1-bit 存储单元：写使能有效时，输入 D 可以影响输出 Q；写使能无效时，反馈通路维持原来的 Q。

写使能	数据输入	输出行为
有效	0 或 1	输出跟随输入变化
无效	任意	输出保持原值

例如一个“按住才能设置”的电路中，enable 为 1 时，D=1 会把锁存器写成 1；enable 回到 0 后，即使 D 变回 0，Q 仍保持 1。这样，过去某一时刻的输入就被硬件保存下来了。

锁存器的问题在于透明窗口。只要 enable 有效，输入变化就可能一路穿透到输出。若多个锁存器共用同一个 enable，并且它们之间还接着组合逻辑，状态可能在一个打开窗口内穿过多级，边界变得模糊。对小电路来说，锁存器易于分析；对大规模同步机器来说，更清晰的做法是把写入压到离散时钟边界。

二、触发器、时钟周期与采样窗口

触发器解决的是“什么时候写入”的问题。它不像锁存器那样在一段时间内透明，而是在时钟沿附近采样输入，把采样值保存到输出。

```
on clock edge:
  Q = D
between clock edges:
  Q holds
```

这让大电路有了清楚节奏。一个时钟沿之后，上一批触发器输出新状态；这些状态经过组合逻辑传播，形成下一批输入；到下一个时钟沿，目标触发器再统一采样。于是“计算”和“保存”被分开了。

最常见的同步路径可以写成：

```
source register -> combinational logic -> destination register
```

时钟沿到来后，源寄存器输出旧状态对应的新值。这个值进入组合逻辑，经过门延迟和布线延迟，最终到达目标寄存器输入。下一个时钟沿到来时，目标寄存器采样这个结果。因此频率不是越高越好。频率提高意味着周期缩短，留给组合逻辑稳定的时间变少。若周期短到最慢路径来不及稳定，目标寄存器就可能采到旧值、过渡值或错误值。

触发器采样也不是数学瞬间。输入 D 必须在时钟沿到来前稳定一小段时间，这称为建立时间；时钟沿之后还要继续稳定一小段时间，这称为保持时间。

约束	含义	违反后果
建立时间	采样前输入要提前稳定	采到错误值或不稳定值
保持时间	采样后输入要继续稳定	新旧值混入采样窗口
传播延迟	上一级输出和组合逻辑需要时间	周期太短会来不及

一条路径的周期账本可以写成：

```
clock period >= source clock-to-Q
                + combinational delay
                + wire delay
                + destination setup time
```

保持时间看起来更小，却同样危险。若源寄存器输出太快、路径太短，目标寄存器可能在同一个时钟沿之后立刻看到新值，从而破坏它本应采旧值的窗口。长路径会限制最高频率，短路径也可能破坏保持约束。

三、关键路径、复位与跨边界输入

一块同步电路里会有很多寄存器到寄存器路径。最长、最慢、最限制周期的那条叫关键路径。它可能是一串加法器，也可能是多级选择器，也可能是远距离布线加上复杂译码。

例如一个 32-bit 串行进位加法器接在两个寄存器之间。最低位进位要一位一位传到最高位，最高位结果最后才稳定。如果把这个加法器放在一个周期里完成，那么周期至少要等最坏进位链走完。若把运算拆成更短的几段，中间插入寄存器，单个周期可以变短，但完成一次完整运算需要更多周期。

结构选择	好处	代价
一周周期完成长运算	控制简单，单条动作周期数少	关键路径长，频率低
拆段插入寄存器	单周期路径短，频率可能高	完成一次动作需要更多边界

触发器通电后的值不一定符合设计预期。若一组状态寄存器随机落在不同值，后面的组合逻辑可能立刻进入非法状态。复位信号的作用，是把关键状态单元带到已知点，例如计数器清零、状态机回到 idle、控制寄存器进入安全默认值。

复位本身也要讲边界。若复位释放时，有的触发器先退出复位，有的后退出复位，状态机可能看到一半新状态、一半旧状态。常见处理是异步拉入复位以便快速进入安全状态，再同步释放，让所有相关触发器在清楚的时钟边界恢复工作。复位不是

“清零所有东西”这么简单。哪些状态需要复位，哪些数据寄存器可以不复位，哪些输出在复位期间必须安全，都要由硬件需求决定。

跨时钟或来自外部世界的输入还会带来亚稳态。若输入 D 正好在时钟沿附近变化，触发器内部节点可能短时间落在不稳定区域，既不像明确 0，也不像明确 1。多数情况下，它会在一小段时间后落回 0 或 1，但落回哪边、需要多久，不能按普通组合逻辑精确保证。因此来自另一个时钟节奏或外部世界的信号，不能直接拿来控制大面积电路，通常要先经过同步结构，降低亚稳态扩散概率。

四、寄存器、计数器与移位结构

一个触发器保存 1 bit。把多个触发器并排，并让它们共享同一个时钟边界，就得到多 bit 寄存器。寄存器可以理解为一组共享时钟和写使能的触发器。若写使能有效，在时钟沿采样输入；若写使能无效，保持原值。

```
on clock edge:
    if write_enable:
        Q = D
    else:
        Q holds
```

寄存器的意义不只是“能存数”。它把连续传播的组合逻辑切成离散步骤。上一个周期的结果在寄存器输出端稳定存在，成为本周期组合逻辑的输入；本周期算出的结果，要到下一个时钟沿才成为新的状态。共享同一提交边界是关键：寄存器保存的是同一个边界上的一组 bit，不是一位一位随意变化。

计数器的下一值来自当前值加一：

```
next = current + 1
on clock edge:
    current = next
```

该结构里有反馈，但反馈不是直接绕回去，而是经过加法器，并且被时钟边界切开。若 current 为 3，本周期组合逻辑算出 next 为 4；只有下一个时钟沿到来，寄存器才保存 4。随后组合逻辑再从 4 算出 5。如果没有寄存器边界，current+1 的结果会立刻回到输入，再加一，再回到输入，电路无法表达“每个周期只增加一次”。

移位寄存器把一组 bit 向左或向右移动。它可以用于串行输入、串行输出、简单延迟线和位级变换。

```
next[3] = current[2]
next[2] = current[1]
next[1] = current[0]
next[0] = serial_in
```

假设 current 为 1010，serial_in 为 1。下一个时钟沿后，寄存器变成 0101。注意，四个位不是按顺序一个一个搬。组合逻辑同时准备好四个 next 输入，时钟沿到来时一起写入。该例说明状态更新不一定是加法，也可以是重新排列、选择、清零、保持或装载外部输入。

五、状态机把硬件动作切成阶段

有限状态机由三部分组成：当前状态、输入条件、下一状态逻辑。当前状态保存在寄存器里，组合逻辑根据当前状态和输入条件决定下一状态。

以一个三阶段硬件控制器为例：

当前状态	条件	下一状态
IDLE	start=1	LOAD
LOAD	ready=1	RUN

RUN	count_done=1	DONE
DONE	clear=1	IDLE

若当前状态是 IDLE 且 start=0, 状态保持 IDLE; 若 start=1, 下一状态逻辑准备 LOAD, 等时钟沿到来后状态寄存器才真正变成 LOAD。状态机因此不会在一个周期里连续越过 IDLE、LOAD、RUN、DONE, 它每次只跨过一个明确边界。

状态机输出可以只由当前状态决定, 也可以同时依赖输入。前者输出更稳定, 后者反应更快但更容易受输入毛刺影响。基本原则是: 状态机不是若干状态名称的集合, 而是“状态寄存器 + 下一状态组合逻辑 + 输出组合逻辑”的硬件结构。

现在硬件已经能够保存一组 bit、按周期加一、按周期移动、按条件切换阶段。下一步要把这些能力用于更大的动作组织: 哪些寄存器在这个周期写入, ALU 做哪种运算, 哪一路数据被选择, 下一阶段去哪里。这就是控制器的基础。

本章的标注对象是“边界”和“状态转移表”。只说某个值会变化不够, 必须说明哪个时钟边界提交, 提交前后的组合路径是什么。

- 纸上实验: 画出一个 D 触发器前后各接一段组合逻辑, 标出当前周期的旧 Q、组合路径上的 D、下一个时钟沿后的新 Q。
- 时序证据: 为一条路径写出 $T_{clk} \geq t_{CQ} + t_{comb} + t_{setup} + t_{skew}$, 再解释每一项对应哪个硬件现象。
- 复位证据: 复位要让控制状态进入已知安全点, 但释放复位也要有同步边界, 不能让状态机半边先跑。
- 状态证据: 寄存器是一组并排触发器; 计数器是寄存器输出经过加法器反馈到输入; 移位寄存器是固定连线选择了相邻 bit。
- 控制证据: 状态机输出可以只依赖状态, 也可以依赖状态和输入; 后者反应快, 但更容易把输入毛刺传给控制线。
- 检查题: 若状态编码里出现未使用编码, 电路上电或受干扰进入它时, 下一状态逻辑应该怎样让机器回到安全状态?

经典系统教材通常会把组合逻辑和状态逻辑分开建模: 组合逻辑计算下一值, 状态元件在边界保存下一值。后面 CPU 的 PC、寄存器、状态机和控制器都遵守这个模型。

运算部件与数据通路

本部分导读

有了稳定状态和组合逻辑，就可以构造算术电路、ALU、标志位、寄存器阵列、内部连线和控制信号。本部分仍然站在纯硬件角度：先让值能够被表示、计算、选择和保存，再讨论可重复动作如何形成。

数值、ALU、数据通路与控制 器

前两章已经解决两个基础问题：bit 如何由电平和开关路径产生，状态如何由触发器和时钟边界保存。本章把第三部分压成一个厚章：固定宽度 bit 如何解释成数，算术和逻辑如何进入 ALU，值如何沿数据通路流动，控制器如何在每个周期给出一组一致的控制信号。

本章的重点是说明 CPU 出现之前的关键部件及其连接方式。到章末，读者应当能走读一条硬件动作：从哪些寄存器读旧值，经哪些选择器进入 ALU，结果如何带着标志位输出，最后由控制器决定是否写回状态。

一、固定宽度 bit 的数值解释

硬件只保存 bit。所谓整数、正负号、溢出，都是对 bit 模式的解释规则。若不先讲清楚表示，后面的加法器、减法器、比较器和 ALU 就会像在做抽象数学；事实上它们处理的始终是固定宽度的一组电线。

4-bit 无符号数 `b3 b2 b1 b0` 的值是：

$$\text{value} = b3 \cdot 8 + b2 \cdot 4 + b1 \cdot 2 + b0$$

因此 `1011` 表示 11，因为它等于 $8+0+2+1$ 。无符号数没有负数概念，所有 bit 都贡献非负权重。4-bit 能表达的范围是 0 到 15，8-bit 能表达的范围是 0 到 255。位宽是硬件限制，不是显示格式。一个 4-bit 加法器只能输出 4 个结果 bit，再加上可能的进位。若 15 加 1，低 4 位回到 `0000`，最高位外产生一个进位。这个进位是无符号运算是否超出范围的证据。

补码表示让负数也能走同一个加法器。以 4-bit 为例，最高位不再只是 8，而是解释成负权重 -8：

$$\text{value} = b3 \cdot (-8) + b2 \cdot 4 + b1 \cdot 2 + b0$$

于是 `1111` 的值是 $-8+4+2+1=-1$ ，`1110` 的值是 -2，`1000` 的值是 -8。4-bit 补码范围是 -8 到 7。求一个数的相反数，可以按“取反加一”理解：

```
x = 0011 // 3
~x = 1100
~x + 1 = 1101 // -3 in 4-bit two's complement
```

这个规则适合硬件，因为取反可以用一排 NOT 或 XOR 门完成，加一可以交给加法器。于是 $a-b$ 可以变成：

$$a - b = a + (\text{NOT } b) + 1$$

同一组 bit 可以按无符号解释，也可以按补码解释。因此溢出判断也不同。

解释方式	关注点	例子
无符号	是否产生最高位外的进位	$15 + 1$ 超出 4-bit 无符号范围
补码有符号	正正得负或负负得正	$7 + 1$ 得到 <code>1000</code> ，被解释为 -8

以下 4-bit 加法给出同一结果在两种解释下的差异：


```
0111 + 0001 = 1000
```

若按无符号解释，这是 $7+1=8$ ，仍在表示范围内。若按补码解释，**0111** 是 7，**0001** 是 1，结果 **1000** 是 -8，正数加正数却得到负数，所以发生有符号溢出。硬件不会替使用者决定哪种解释成立；它只提供证据：结果 bit、进位、符号位变化、溢出标志。

移位器也是算术部件。左移一位通常相当于乘以 2，但前提是移出去的 bit 没有携带需要保留的信息。右移更容易出差别。逻辑右移在高位补 0；算术右移保留符号位，用于补码有符号数。

```
logical right shift: 1000 >> 1 = 0100
arithmetic right shift: 1000 >> 1 = 1100
```

若 **1000** 按 4-bit 补码解释为 -8，算术右移得到 **1100**，也就是 -4；逻辑右移得到 **0100**，也就是 4。两个结果差异巨大。原因不是移位器“出错”，而是控制信号选择了不同硬件规则。

二、ALU 把候选计算收进一个组合核心

ALU 是算术逻辑单元。它不是孤立出现的大型部件，而是因为加法、减法、按位逻辑、移位和比较反复出现，硬件需要把这些能力集中到一个受控制的组合逻辑块中。ALU 的本质是：多个候选计算电路并存存在，控制信号选择本周期采用的结果。

一个最小 ALU 可以写成：

```
ALU(a, b, op) -> result, flags
```

a 和 b 是两个操作数，op 是选择信号，result 是结果，flags 是结果的附带证据。

op	运算	硬件来源
ADD	$a + b$	加法器
SUB	$a - b$	加法器加反相控制
AND	$a \& b$	按位 AND 门阵列
OR	$a b$	按位 OR 门阵列
XOR	$a \text{ xor } b$	按位 XOR 门阵列
SHIFT	移位	移位器

ALU 里最值得细看的是 SUB。若单独做减法器，会增加面积和连线。补码让它复用 ADD 的加法器：

```
if op = ADD:
    b_to_adder = b
    carry_in = 0
if op = SUB:
    b_to_adder = NOT b
    carry_in = 1
result = a + b_to_adder + carry_in
```

实际电路里，b 的每一位前面可以接 XOR 门：当 sub 控制为 0，b 原样通过；当 sub 控制为 1，b 被取反。最低位进位输入也由 sub 控制。这样 ADD 和 SUB 共用同一条进位链。这个案例展示了硬件设计的常见思路：先找到数学规则之间的共用部分，再用少量控制信号复用同一块组合逻辑。

ALU 内部可以并行产生多个候选结果，再用多路选择器按 op 选出最终 result。

```
add_result
and_result
```



```

or_result
shift_result
result = mux(op, add_result, and_result, or_result, shift_result)

```

这不表示所有运算都“按顺序算一遍”。组合逻辑是并行传播的：加法器、按位逻辑、移位器都可能同时产生自己的候选输出。最终哪一路被后级看见，由 mux 决定。代价也随之出现。候选电路越多，ALU 面积越大；mux 输入越多，选择延迟越大；若某个候选结果来自长进位链，它可能成为 ALU 的关键路径。

ALU 常见标志位包括：

标志位	含义	用途
Zero	result 是否为 0	相等判断、条件选择
Carry	无符号进位或借位证据	多字长加法、无符号比较
Overflow	补码有符号溢出	有符号范围检查
Negative	result 最高位是否为 1	补码符号证据

Zero 可以由所有结果 bit 做 OR 后再取反得到。Negative 通常直接来自结果最高位。Carry 来自加法器最高位外的进位。Overflow 则要看有符号加减法中符号是否出现不可能的变化。标志位不是额外装饰，而是后续控制的重要输入。例如比较 A 和 B 是否相等，可以让 ALU 执行 A-B；若结果所有 bit 都是 0，Zero=1，控制器就知道两个输入相等。

三、数据通路让值从旧状态走到新状态

数据通路回答一个问题：值从哪里来，经过哪里，最后保存到哪里。孤立的寄存器和 ALU 还不能完成连续动作。必须把寄存器阵列、选择器、ALU、内部连线和写回路接成一个可控制的闭环。

一个寄存器只能保存一个值。若硬件需要同时保存多个中间值，就需要寄存器阵列。寄存器阵列可以理解为一组编号寄存器，加上读写选择电路。

端口	作用	类型
读地址 A/B	选择两个源寄存器	输入
读数据 A/B	输出两个源值	输出
写地址	选择目标寄存器	输入
写数据	提供要保存的新值	输入
写使能	决定是否在时钟沿写入	输入

读地址进入译码和选择网络，决定哪两个寄存器的值出现在读数据端口。写地址进入译码器，写使能有效时，只允许被选中的那个寄存器在时钟沿接收写数据。这样，一组寄存器就能被编号访问，而不是每个寄存器单独拉一套控制线。

ALU 的两个输入不一定总来自同一处。一个输入可能来自寄存器阵列，也可能来自 PC、计数器或固定 0；另一个输入可能来自寄存器、常量或扩展后的字段。因此 ALU 输入前通常有 mux。

```

alu_a = mux(a_sel, reg_a, counter_value, zero)
alu_b = mux(b_sel, reg_b, constant, offset_value)

```

以寄存器 R1 和 R2 相加、结果写入 R3 为例。数据通路需要让读地址 A 选择 R1，读地址 B 选择 R2；a_sel 选择 reg_a，b_sel 选择 reg_b；ALU op 选择 ADD。此时 ALU 输出就是 R1+R2。

写回数据也不一定只来自 ALU。它可能来自 ALU，也可能来自存储器读口、外部输入或 PC+4。写回前也需要 mux。

```

write_data = mux(wb_sel, alu_result, memory_data, input_data)

```

在 R1+R2 写入 R3 的动作中, wb_sel 选择 alu_result, 写地址选择 R3, reg_write 为 1。等时钟沿到来, R3 才真正保存结果。若 reg_write 为 0, 即使 ALU 算出了结果, 寄存器阵列也不会改变。

当多个部件都可能把值送到同一个目的地时, 不能让它们同时强行驱动同一条线。要么用 mux 明确选择一路, 要么用总线协议规定同一时刻只有一个驱动者。ALU 输出、存储器输出和外部输入如果直接接在一起, 一旦两个源同时输出不同电平, 就会形成冲突。mux 让这件事变成明确选择: 当前周期只允许一路成为 write_data。

一个最小数据通路闭环可以写成:

```
register file -> mux -> ALU -> mux -> register file
```

它能在一个周期内读取旧值、选择输入、完成组合计算, 并在时钟沿保存结果。这个闭环已经能表达很多动作, 但它还不会自己决定“当前周期做什么”。哪些地址送入寄存器阵列, ALU 做加法还是减法, 写回选择哪一路, 写使能是否打开, 都需要控制器统一产生。

四、控制器输出一组一致的控制线

数据通路提供很多可能路径, 但同一周期不能所有路径都随意打开。控制器的任务是根据当前状态和外部条件, 产生一组控制信号, 让数据通路执行一个明确动作。控制器不是写在纸上的命令列表, 而是一块输出控制电线的硬件。

控制信号通常是一组 0/1 或多 bit 选择值:

```
alu_op
a_sel
b_sel
wb_sel
reg_write
mem_read
mem_write
next_state_sel
```

这些信号直接接到 ALU、mux、寄存器阵列和存储器接口。控制信号给错, 数据通路就会走错。例如 reg_write 本应为 0 却变成 1, 某个寄存器会在时钟沿被意外覆盖; wb_sel 本应选择 ALU, 却选了外部输入, 写回值就会错。

若动作规则简单, 可以用组合逻辑直接从输入生成控制信号。例如输入 mode 为 0 时做加法, 为 1 时做按位 AND:

```
mode = 0 -> alu_op = ADD
mode = 1 -> alu_op = AND
```

硬连线控制速度快, 结构也直接。缺点是规则复杂后, 组合逻辑会变大, 修改一个动作可能影响多条控制线。若一个动作无法在一个周期内完成, 就需要状态机。状态机保存“当前走到哪一步”, 每个状态输出一组控制信号, 并决定下一状态。

状态	控制动作	下一状态
S0	选择源寄存器, 锁存操作数	S1
S1	ALU 计算, 结果进入临时寄存器	S2
S2	选择临时结果, 写回目标寄存器	S0

表中每一行都对应真实硬件动作。S0 打开读路径和临时寄存器写使能; S1 设置 ALU 输入和 op, 并保存 ALU 结果; S2 打开写回路路径和目标寄存器写使能。状态机把复杂动作分成多个清楚边界, 避免把所有组合逻辑放入一个过长周期。

五、用控制 trace 走读一次动作

复杂动作可以拆成微操作：读、选择、计算、保存、更新状态。本节所称微操作，指一个周期内同时打开的一组硬件控制信号。以下以 R1 和 R2 相加、结果写入 R3 为例。

```
S0: read_sel_a=R1, read_sel_b=R2
    A_write=1, B_write=1
    reg_write=0, ALU_out_write=0

S1: a_sel=A, b_sel=B, alu_op=ADD
    ALU_out_write=1
    reg_write=0, A_write=0, B_write=0

S2: wb_sel=ALU_out, write_sel=R3
    reg_write=1
    A_write=0, B_write=0, ALU_out_write=0
```

第一步 S0 需要落实为具体控制信号。read_sel_a 选中 R1 后，寄存器阵列的第一个读端口会把 R1 当前保存的 bit 组合驱动到读数据线上；read_sel_b 选中 R2 后，第二个读端口同理输出 R2。等这些组合路径稳定后，时钟沿到来，A_write 和 B_write 让 A、B 锁存器保存这两个值。这个周期里 reg_write 必须为 0，因为此时写回 mux 上的值并不代表最终结果。

第二步 S1 负责 ALU 计算。A、B 锁存器的输出作为 ALU 输入，alu_op=ADD 让 ALU 内部选择加法路径。加法电路不是瞬间得到稳定结果，低位进位会影响高位，输出线上可能经历一小段变化过程。控制器在这个周期打开 ALU_out_write，让时钟沿把已经稳定的 ALU 输出锁存下来；同时 reg_write 仍然保持 0。

第三步 S2 负责写回。wb_sel 选择 ALU_out 锁存器，write_sel 选择 R3，reg_write 在这个周期置 1。到时钟沿时，寄存器阵列把写数据端口上的值保存到 R3。至此，R1 和 R2 没有被改动，R3 得到 R1+R2 的结果，控制器回到空闲或取下一动作的状态。

该例也说明控制错误的风险。若 S2 的 wb_sel 错选成 B 锁存器，R3 写入的就是 R2，而不是相加结果；若 S1 提前打开 reg_write，R3 可能在 ALU 输出尚未稳定时被覆盖；若 S0 忘记关闭 ALU_out_write，旧的计算边界会被无关读数污染。微操作表的价值在于：每一步都能检查哪些线打开、哪些线关闭、哪个状态元件会在时钟沿改变。

本章的经典读法是 trace：从 bit 解释，到 ALU 候选计算，到数据通路值流，再到控制信号时间表。

- 位级证据：补码取负可写成按位取反再加一；减法可复用加法器执行 $a + (\sim b) + 1$ 。
- 标志证据：Carry 适合无符号边界，Overflow 适合补码有符号边界；不能把两者混成同一个“溢出”。
- ALU 证据：候选计算并行产生，选择器选出一个结果，标志位保存结果证据。
- 数据通路证据：为 $R3 = R1 + R2$ 写读地址、ALU 输入选择、alu_op、写回选择、写地址、写使能。
- 控制证据：每个周期哪些写使能为 1，哪些 mux 选择哪一路，ALU 做什么，都要能列成 trace。

- 检查题：若 ALU 结果已经正确，但写回 mux 选择线接错，寄存器结果会怎样？排查时应看 ALU，还是写回路径和控制器？

经典教材会把控制器和数据通路并排讲：数据通路说明“能走哪些路”，控制器说明“本周期允许哪条路”。两者合起来，才是可执行动作。

最小 CPU 与整机硬件

本部分导读

前面已经有了计算、状态、选择和控制。现在把它们合成 CPU，再继续向外连接主存、总线、设备、中断控制器和启动硬件，并通过 8086、80386 等经典芯片观察真实处理器的演进。

CPU、经典芯片与整机硬件

前面已经具备构造处理器的必要部件：组合逻辑、状态元件、时钟边界、ALU、数据通路和控制器。现在可以把它们合成为 CPU，并进一步连接主存、总线、设备和启动硬件，形成整机。本章不只停留在“最小 CPU”模型，还引入 8086、80386 等经典处理器，说明真实芯片如何在位宽、地址空间、总线接口、保护机制和系统结构上逐步扩展。

CPU 可以视为一组同步硬件闭环：PC 指出下一条编码，取指路径取回指令 bit，译码逻辑生成控制信号，数据通路读取和计算，状态元件在时钟边界提交结果。经典芯片的差异，正是这些闭环在位宽、编码、寻址、控制复杂度和外部接口上的不同取舍。

一、从动作编码到最小 CPU

若希望同一套数据通路在不同时间做不同动作，就需要把“这一次做什么”也表示成 bit。动作编码可以先写成最小形式：

```
opcode | dst | src_a | src_b
```

`opcode` 表示动作类型，例如 ADD、SUB、AND；`dst` 表示写入哪个寄存器；`src_a` 和 `src_b` 表示读取哪两个寄存器。控制器读取这些字段后，把它们译成 ALU op、读地址、写地址、写使能和写回选择。编码本身仍然是硬件输入，不是文字命令。

若有一组动作编码连续放在存储结构中，硬件还需要知道当前取哪一条。这个位置由 PC 保存。PC 本质上也是寄存器，只是它保存的是“下一条动作编码的地址”。

部件	作用
PC	保存下一条编码地址
指令存储器	根据 PC 输出当前编码
控制器	根据编码字段产生控制信号
寄存器阵列	保存通用数据
ALU	做整数和逻辑运算
选择器网络	选择 ALU 输入、写回来源和下一 PC

顺序执行时，下一 PC 可以由加法器产生：

```
pc_next = pc_current + instruction_size
```

如果每条编码占 4 个字节，`instruction_size` 就是 4；如果是可变长度指令，下一 PC 的计算会更复杂。关键事实是：顺序执行不是抽象的“下一行”，而是 PC 这个状态寄存器保存了新的地址。

最小单周期模型可以这样走：

阶段	硬件动作	边界
取指	PC 送入指令存储器，得到编码	组合路径
译码	控制器产生 mux、ALU、写使能信号	组合路径
执行	寄存器阵列读值，ALU 计算	组合路径
写回	结果在时钟沿写入寄存器或 PC	状态边界

用一条普通运算编码走读：当前位置 PC=100，指令存储器在地址 100 处保存的

是：

```
ADD dst=R3, src_a=R1, src_b=R2
```

周期开始时，PC 输出稳定为 100。这个地址进入指令存储器，存储器经过地址译码后把这条编码输出。控制器看到 opcode=ADD，于是把 alu_op 设为 ADD，把寄存器阵列的两个读地址分别设为 R1 和 R2，把写地址设为 R3，把 wb_sel 设为 ALU，把 reg_write 设为 1，同时让下一 PC 选择 PC 加指令长度。

这些控制信号稳定后，寄存器阵列的两个读端口输出 R1 和 R2 当前的值。它们经过 ALU 输入选择器进入 ALU，加法路径产生结果。结果再经过写回选择器，到达寄存器阵列的写数据端口。另一边，PC 加法器准备下一条指令地址，PC 选择器把它送到 PC 的下一值输入。只有到时钟沿，两个状态变化才同时提交：R3 保存 ALU 结果，PC 保存下一条地址。

分支也可以归结为下一 PC 选择。顺序情况下，下一 PC 是 PC 加指令长度；条件分支则由 ALU 比较结果和选择器决定下一 PC 来源：

```
branch_taken = condition_met AND branch_op
next_pc = mux(branch_taken, pc_sequential, branch_target)
```

分支的硬件本质是改变 PC 这个寄存器的下一值。下一周期取到哪条编码，完全由这个新 PC 决定。

二、8086：从芯片接口看早期微处理器

8086 是理解早期 x86 体系的重要起点。它是 16-bit 微处理器，拥有 16-bit 通用寄存器和 16-bit 数据通路特征，同时通过 20-bit 地址能力访问最多 1 MiB 物理地址空间。它的历史意义不只在指令集，还在于展示了 CPU 芯片如何通过外部总线、地址锁存、控制信号和存储器/外设共同构成一台机器。

8086 的寄存器组包括 AX、BX、CX、DX、SP、BP、SI、DI，以及 CS、DS、SS、ES 等段寄存器。段寄存器和偏移地址共同形成物理地址：

```
physical_address = segment * 16 + offset
```

这个规则反映了一个时代的硬件取舍：内部仍以 16-bit 编程模型为主，但外部需要访问超过 64 KiB 的地址空间，于是通过段基址左移和偏移相加得到 20-bit 地址。分段不是高级抽象，而是地址形成硬件的一部分。

8086 的外部总线也值得注意。地址线和数据线存在复用，某些引脚在总线周期的不同阶段承担不同含义。硬件需要地址锁存器在地址有效阶段保存地址，随后这些引脚可以用于数据传输。一次总线访问不是“读内存”三个字，而是分阶段发生的事务：输出地址、锁存地址、给出读写控制、等待存储器或 I/O 响应、传输数据、结束总线周期。

8086 观察点	硬件含义
16-bit 寄存器	数据运算和编程模型以 16-bit 为核心
20-bit 地址形成	通过段寄存器和偏移访问 1 MiB 物理空间
地址/数据复用总线	芯片引脚有限，外部需要锁存和分阶段传输
存储器与 I/O 周期	访问目标由控制信号和地址空间规则区分
BIU/EU 分工	取指队列和执行部件可部分重叠工作

8086 内部常被划分为总线接口单元和执行单元。总线接口单元负责取指、形成物理地址、访问外部总线，并维护预取队列；执行单元负责译码和执行指令。这个分工说明，即使没有现代深流水，CPU 内部也已经在尝试把“取指访问外部世界”和“执行内部运算”分开，以减少外部存储访问延迟对执行的影响。

从本书前面建立的硬件模型看，8086 可以视为一个更复杂的同步系统：PC 在 x86 语境中体现为指令指针和代码段组合；地址形成部件把段和偏移送入加法路径；

总线接口把地址和控制信号变成外部事务；执行部件译码指令并驱动寄存器、ALU 和标志位。8086 不是脱离前面章节的新对象，而是前述部件在真实芯片中的组合。

三、80386：32-bit、保护模式与地址转换

80386 把 x86 推入 32-bit 时代。它扩大了通用寄存器宽度，引入 EAX、EBX、ECX、EDX、ESP、EBP、ESI、EDI 等 32-bit 形式；地址和数据处理能力显著增强，并提供保护模式、分页支持和更完整的内存保护机制。相较 8086，80386 展示的不只是“位宽变大”，而是 CPU 开始承担更复杂的系统级硬件职责。

80386 的一个关键变化是 32-bit 线性地址。程序形成的有效地址先经过分段机制得到线性地址；在启用分页时，线性地址再经过页表转换为物理地址。可以用简化链路表示：

```
effective address -> segmentation -> linear address
linear address    -> paging          -> physical address
```

这条链路说明，地址不再只是“寄存器加偏移后送到总线”。CPU 内部必须有地址形成、权限检查、页表访问、异常触发等硬件机制。若页表项不存在、权限不允许、访问越界，CPU 不是简单得到一个错误数据，而是产生异常事件，控制流转入规定入口。

保护模式把“谁能访问什么”变成硬件可检查的合同。段描述符、特权级、页表权限等机制，使得不同程序或系统组件不能随意访问全部内存或执行全部指令。这些机制后来成为现代操作系统隔离、虚拟内存和系统调用的硬件基础。对本书来说，重点不是展开操作系统，而是看见硬件层面的扩展：CPU 不只执行 ALU 运算，还参与地址翻译、权限判断和异常入口选择。

80386 观察点	硬件含义
32-bit 通用寄存器	数据通路和编程模型扩展到 32-bit
32-bit 线性地址	地址空间显著扩大，地址形成更复杂
保护模式	CPU 硬件检查权限、界限和特权级
分页机制	线性地址可经页表映射到物理地址
异常机制	地址或权限错误进入硬件规定的控制入口

从 8086 到 80386，可以看到 CPU 和整机边界的变化。8086 主要展示了早期微处理器如何通过外部总线接入存储器和 I/O；80386 则展示了处理器如何把内存管理和保护机制纳入芯片内部。后来的处理器继续把 cache、TLB、流水线、分支预测、乱序执行、多核互连等机制纳入芯片，但基本问题没有改变：地址从哪里来，权限如何检查，数据经哪条路径返回，状态在哪个边界提交。

四、主存、总线、I/O 与 DMA

寄存器阵列很快，但容量有限。主存提供大容量状态空间，用地址选择位置，用数据线传递内容，用控制信号说明读还是写。

信号	作用
地址	指出访问哪个存储位置
写数据	写入时提供数据
读数据	读取时返回数据
读写控制	指明本次访问方向
ready/valid	指明请求或响应是否有效

一次读访问可以这样理解：CPU 给出地址和读控制，主存经过内部译码、阵列访问和输出驱动后，把数据放到读数据线上，并用响应信号表示结果有效。一次写访问则是 CPU 同时给出地址、写数据和写控制，主存在合适边界把数据保存到对应位

置。

CPU 内部一个周期可以很短，而主存访问可能慢得多。若每次取指、读数据、写数据都等主存完成，ALU、寄存器阵列和控制器会长时间空转。层级存储的动机，是把最近、最常用的一小部分内容放到更近、更快的位置。cache 可以先理解成靠近 CPU 的小型硬件存储，它通过 tag、data、valid、dirty 等字段完成命中判断、数据返回、替换和写回。

cache 字段	作用
tag	判断当前地址是否对应这一项
data	保存从主存取来的数据块
valid	表示这一项是否有效
dirty	表示这一项是否被改过、尚未写回下层

当 CPU、cache、主存、设备控制器都要通信时，需要互连结构。简单系统可以用共享总线，复杂系统会用分层互连或片上网络。无论形式如何，它至少要解决五个问题：谁发起请求，地址指向哪个目标，数据往哪边走，多个发起者冲突时谁先走，响应怎样回到发起者。

设备控制器通常把自己的控制寄存器映射到地址空间中。CPU 对某些地址读写，实际访问的是设备寄存器，这就是 MMIO。写一个控制寄存器可能启动设备动作，读一个状态寄存器可能得到设备是否完成、是否出错、缓冲区是否可用。设备寄存器虽然像内存一样通过地址访问，但语义由设备控制器定义。

若设备要搬运大量数据，由 CPU 逐字读写会占用大量内部数据通路时间。DMA 控制器可以在获得总线使用权后，直接在设备和主存之间搬运数据。CPU 只需要配置源地址、目标地址、长度和方向，真正的数据搬运由 DMA 硬件完成。DMA 也带来新的边界：CPU 和 DMA 都可能访问主存，互连必须仲裁；cache 可能保存旧副本，系统必须规定 DMA 区域怎样与 cache 保持一致；设备完成后还需要可观察的完成信号。

设备完成工作或状态变化时，需要把事件送回 CPU。中断控制器负责收集多个设备的请求，进行屏蔽、优先级排序和投递。站在硬件角度，中断是一条事件入口：外部设备把请求线送进控制结构，CPU 在规定边界看到这个请求，并转入对应处理路径。

五、整机启动与经典芯片后的演进

通电或复位后，CPU、主存控制器、互连和设备控制器都必须进入规定初始状态。PC 通常被置到固定启动地址，那里连接 ROM、片上存储或其他启动介质。启动路径的第一件事，是让硬件从可预测位置开始取编码。

整机启动不仅是 CPU 复位。时钟源要稳定，复位树要按顺序释放，主存控制器要完成训练或初始化，互连要进入可转发请求的状态，必要设备要处于安全默认值。若这些硬件条件没有满足，即使 CPU 本身能取指，也可能在第一次访问主存或设备时无法继续推进。

经典微处理器的发展可以用若干硬件剖面概括。下表不是处理器史年表，而是说明每一代芯片把哪些硬件问题推到读者面前。

芯片或系列	典型特征	对硬件学习的意义
4004	4-bit 微处理器	微处理器把控制器、寄存器和算术部件集成到单芯片形态
8080/8085	8-bit 微处理器	外部总线、地址空间、存储器和 I/O 开始形成通用微机结构
8086/8088	16-bit x86 起点	段：偏移地址、指令预取、复用总线和外部锁存器体现早期整机取舍

80286	保护模式前驱	分段保护和特权机制开始进入处理器硬件合同
80386	32-bit x86	32-bit 寄存器、线性地址、分页和异常机制把 CPU 推向系统级硬件
80486/Pentium	片上 cache 与更深执行结构	流水线、cache、浮点单元和更复杂控制逻辑成为性能核心

从 8086、80386 向后看，处理器继续沿几条方向扩展。第一，内部执行路径更深，流水线把取指、译码、执行、访存、写回切成多个阶段。第二，存储层级更复杂，片上 cache 和 TLB 成为地址访问性能的关键。第三，控制逻辑更复杂，分支预测、乱序执行和异常恢复要求硬件保存更多内部状态。第四，多核和片上互连让“总线事务”扩展成核间一致性和缓存一致性协议。第五，外设从简单寄存器发展到 PCI、PCIe、USB、SATA 等分层协议，但本质仍然是地址、数据、控制、握手、完成和错误状态。

演进方向	保持不变的硬件问题
流水线	状态边界如何切分，错误路径如何清除
cache/TLB	地址如何命中、缺失、替换和回填
保护与异常	何时拒绝访问，如何进入规定处理入口
DMA 与设备	谁发起事务，谁完成事务，完成如何通知 CPU
多核一致性	多个观察者如何看到受控顺序的内存结果

到这里，本书的硬件链路闭合：电和器件形成可控开关，开关形成门，门形成组合逻辑，反馈和时钟形成状态，寄存器和 ALU 形成数据通路，控制器组织动作，CPU 执行编码，主存、cache、互连、设备和启动路径把它扩展成整机。8086 和 80386 的意义在于提供了两个清晰历史剖面：一个展示早期微处理器如何连接总线和地址空间，一个展示处理器如何承担更复杂的地址转换、保护和异常机制。

整机硬件的标注对象是事务和边界。一次访问不是“读内存”三个字，而是地址、方向、数据、发起者、目标、握手、响应和错误状态组成的一次硬件事务。

- 纸上实验：走读一条 ADD 指令、一条条件分支、一次 cache 命中读、一次 cache 未命中读、一次 MMIO 写设备命令、一次 DMA 完成并发中断。
- 经典芯片证据：8086 用段寄存器和偏移形成 20-bit 物理地址；80386 引入 32-bit 地址、保护模式和分页机制。
- 机器证据：主存、cache、设备寄存器都可以被地址访问，但语义不同；地址译码决定请求落到哪类目标。
- 并发证据：DMA 和 CPU 都能成为总线发起者，必须有仲裁；cache 和 DMA 共享主存时，必须有一致性或同步规则。
- 检查题：CPU 写设备命令寄存器后，能否立刻假设设备动作完成？应当看 MMIO 写完成、设备 done 位、中断，还是 DMA completion？

经典系统教材会把整机看成多层抽象：指令看到寄存器和地址，硬件看到事务，设备看到寄存器和状态机，系统软件最终看到完成事件。能把这些层对应起来，才算从最小 CPU 走到了真正机器。

读完后的检查表

读完本书后，至少应该能回答下面这些问题：

主题	能力要求
比特	能说明为什么计算机用两个稳定区间表示 0 和 1，而不是追求连续电压的精确值。
逻辑门	能从开关路径解释 NOT、AND、OR、NAND 的行为，并理解为什么 NAND 可以构造其他门。
组合逻辑	能把真值表转成门电路，解释半加器、全加器、多路选择器和译码器分别解决什么问题。
时序逻辑	能说明锁存器、触发器、寄存器和时钟边界的关系，知道为什么组合逻辑不能保存历史。
状态机	能把状态、输入、转移条件和输出分开，并说明状态机怎样控制一组硬件动作。
ALU	能把加法、减法、按位运算、比较和标志位放在同一个计算部件里解释。
最小 CPU	能画出 PC、指令存储器、寄存器堆、ALU、数据存储器 and 控制器之间的数据流。
整机硬件	能说明主存、总线、MMIO、DMA、中断控制器和复位启动怎样把最小 CPU 扩展成整机。

最终检查不以“能不能复述章节标题”为准，而以能不能给出机器证据为准。

- 给一个信号节点，能说出它何时被拉高、何时被拉低、何时悬空、何时冲突。
- 给一个组合电路，能列真值表、最长路径、可能毛刺和后级采样边界。
- 给一个状态电路，能指出旧状态、下一状态、时钟沿、建立/保持约束和复位行为。
- 给一个算术结果，能分别按无符号和补码解释，并说明 Carry 与 Overflow 的差别。
- 给一条最小 CPU 指令，能写出控制信号 trace、数据通路 trace 和 PC 下一值。
- 给一次内存或设备访问，能写出地址译码、总线/互连事务、响应返回和完成证据。

如果这些证据能写出来，说明读者已经把“电平 -> 门 -> 组合逻辑 -> 状态 -> 数据通路 -> 控制器 -> CPU -> 整机”的链路连成了一台机器。